Documentación Técnica: Proyecto CE2048

Introducción

La presente documentación técnica detalla el diseño, la arquitectura y la implementación del proyecto CE2048, desarrollado con el propósito de aplicar y reafirmar de manera práctica los conceptos fundamentales del paradigma de programación funcional utilizando el lenguaje Racket (Scheme). A través de la construcción de una réplica interactiva del popular videojuego 2048, este proyecto explora la manipulación profunda de estructuras de datos basadas en listas bidimensionales (matrices), el uso intensivo de la recursividad pura y el manejo de estados sin mutación imperativa en su motor lógico.

Adicionalmente, el proyecto trasciende la ejecución en consola al integrar una interfaz gráfica interactiva desarrollada con la biblioteca "racket/gui/base". Esta documentación desglosa cómo se logró acoplar exitosamente un motor matemático estrictamente funcional con un entorno visual dinámico, permitiendo animaciones fluidas, validación de entradas y la personalización del tamaño del tablero de juego.

1.1. Descripción de las funciones implementadas

Módulo de Lógica del Juego (logica.rkt)

Este módulo encapsula todo el motor matemático y las reglas fundamentales del juego 2048. Su construcción se basó estrictamente en el paradigma funcional, operando mediante listas de listas y recursividad profunda sin el uso de iteradores tradicionales o mutación de estado.

- **crear-fila / crear-matriz:** Estas funciones son las encargadas de inicializar la estructura de datos del tablero. crear-fila utiliza recursividad de cola (controlada por decrementos matemáticos) para ensamblar una lista plana de ceros. Posteriormente, crear-matriz invoca a crear-fila repetidas veces para construir una lista de listas, lo que conforma la abstracción de una cuadrícula bidimensional de tamaño variable (n x m).
- **buscar-en-fila / buscar-x:** Representan el motor de búsqueda transversal de la matriz. buscar-en-fila evalúa elemento por elemento utilizando la directiva equal?, deteniéndose y retornando verdadero al primer hallazgo. buscar-x escala este comportamiento iterando sobre cada fila de la matriz global, siendo fundamental para verificar la presencia de ceros (espacios vacíos) o la ficha ganadora (2048).
- **adyacentes-en-fila? / adyacentes-en-matriz?:** Funciones booleanas de validación de estado. Analizan matemáticamente si existen dos valores idénticos en posiciones contiguas. Se reescribieron utilizando exclusivamente estructuras cond anidadas para

evadir el uso de operadores lógicos como or, evaluando si una fila está vacía, si posee un solo elemento, o si el primer elemento equivale al segundo (utilizando composición de car y cdr).

- **sin-movimientos?:** Esta función evalúa tres escenarios críticos mediante una cascada de condiciones lógicas estrictas: la existencia de espacios en blanco, la posibilidad de fusiones horizontales y la posibilidad de fusiones verticales (evaluadas mediante la transposición de la matriz). Si ninguna condición se cumple, decreta el fin del juego.
- **ganaste?:** Función de validación de victoria que escanea la totalidad de la estructura matricial en busca del entero 2048, indicando la culminación exitosa del objetivo principal.
- **extraer-elemento / extraer-mat:** Sistema de acceso a coordenadas cartesianas dentro de una estructura funcional pura. Navegan descartando recursivamente las cabezas de las listas (cdr) hasta alcanzar el índice deseado, permitiendo leer el estado de una casilla en específico (fila, columna) para su posterior renderizado gráfico.
- **aplastar-matriz / contar-ceros:** Funciones de aplanamiento y conteo. aplastar-matriz concatena recursivamente todas las filas utilizando append para convertir la estructura bidimensional en un vector unidimensional. Esto permite a contar-ceros obtener la cantidad exacta y absoluta de casillas disponibles para insertar nuevas fichas.
- **generar-numero / reemplazar-n-cero:** Funciones de inserción probabilística estocástica. generar-numero utiliza el reloj del sistema para devolver un 4 (10% de probabilidad) o un 2 (90%). reemplazar-n-cero recorre la lista aplanada llevando un contador interno y sustituye exclusivamente el espacio vacío que coincide con el índice aleatorio calculado.
- **tomar-m / quitar-m / reconstruir-matriz:** Conjunto de funciones responsables de devolver la matriz a su topología bidimensional original tras el proceso de aplanamiento e inserción. Cortan la lista unidimensional en fragmentos de tamaño m y los reagrupan jerárquicamente utilizando cons.
- **insertar-una-segura / insertar-dos-casillas:** Wrappers de alta jerarquía. Previenen errores fatales de ejecución (como desbordamientos por matrices llenas) garantizando que la secuencia de aplanar-insertar-reconstruir solo se ejecute si existen ceros disponibles en el tablero.
- **quitar-ceros / sumar-fila / rellenar-ceros:** Estas tres funciones conforman el núcleo de la física del juego. quitar-ceros desplaza los números hacia un extremo simulando el empuje. sumar-fila fusiona los números adyacentes iguales. Finalmente, rellenar-ceros restaura la longitud de la lista para que la matriz no pierda su forma geométrica.
- **mover-fila-izq:** Función orquestadora que encadena la secuencia física completa (quitar, sumar, rellenar) aplicando el concepto de tuberías (pipelines) sobre una fila individual.
- **sacar-1f / borrar-1f / transpuesta:** Operaciones de álgebra matricial. Convierten las filas de la matriz en columnas mediante la extracción iterativa de las cabezas de cada sublista. Esta abstracción geométrica es vital para procesar movimientos verticales.

- **reversa-lista / reversa-matriz:** Funciones de inversión en espejo. Voltean el orden de los elementos permitiendo reutilizar la lógica de colisión izquierda para efectuar movimientos hacia la derecha.
- **mover-izq-base / mover-der-base / mover-arriba-base / mover-abajo-base:** Funciones matemáticas puras de desplazamiento. Aplican la transformación de mover-fila-izq a toda la estructura realizando las combinaciones necesarias de rotación (transposición) e inversión (reversa) según la dirección vectorial solicitada.
- **mover-izq / mover-der / mover-arriba / mover-abajo:** Funciones de control de estado por turno. Analizan si el tablero sufrió mutaciones matemáticas tras aplicar el movimiento base. En caso de haber un cambio estructural, autorizan la inyección de una nueva pieza al tablero, cumpliendo con la regla estándar de 2048.

Módulo de Interfaz Gráfica (juego.rkt)

Este módulo se encarga de conectar la lógica matemática pura con el usuario a través de un entorno visual interactivo, manejando el estado de la aplicación, las entradas del teclado y los ciclos de dibujado mediante la librería racket/gui/base.

- **validar-entrada:** Intercepta las cadenas de texto digitadas por el usuario en las cajas de configuración de tamaño y aplica un proceso de filtrado estricto. Verifica que la conversión a número sea exitosa, rechaza decimales, y asegura que las dimensiones del tablero se mantengan en un rango jugable y seguro para la memoria (entre 4 y 10).
- **obtener-fila / obtener-col / calcular-destinos-linea / generar-matriz-destinos:** Sistema predictivo de coordenadas para el motor de animaciones. En lugar de mover las imágenes a ciegas, estas funciones precalculan matemáticamente en qué índice de la matriz debe terminar cada ficha individual tras ocurrir colisiones y sumas. Esto permite que el renderizado gráfico interpole las posiciones (X, Y) suavemente sin que las fichas se sobrepongan de manera irreal en los bordes.
- **obtener-color-fondo / obtener-color-texto:** Mapeadores de hojas de estilo. Contienen las definiciones de la paleta de colores RGB oficial del juego 2048, asignando tonos cálidos progresivos dependiendo de la magnitud numérica de la ficha evaluada.
- **ventana-juego / panel-superior / btn-opciones:** Componentes estructurales instanciados de la programación orientada a objetos de la GUI de Racket. Conforman el contenedor principal de la aplicación, el área de controles superiores y el disparador del menú de reinicio y configuración de dimensiones.
- **timer-animacion:** Temporizador asíncrono que interrumpe la ejecución lineal para redibujar el tablero repetidas veces (cuadros por segundo). Modifica la variable global anim-frames hasta llegar a su límite, punto en el cual inserta la pieza nueva, detiene el ciclo visual, y permite que el código evalúe las condiciones de fin de juego.
- **mi-canvas% / lienzo:** El lienzo interactivo subclasificado. Sobrescribe el método de eventos de teclado (on-char) para ignorar entradas mientras hay animaciones en curso. Posee una compleja función de redibujado matricial iterativo que calcula posiciones matemáticas en pantalla, aplica márgenes dinámicos y dibuja rectángulos redondeados

con fuentes responsivas dependiendo de los límites vectoriales del tablero y el progreso de la animación.

- **mostrar-victoria / mostrar-derrota:** Gestores de interrupción de flujo mediante cajas de diálogo contextuales (message-box/custom). Alertan al usuario del fin de la partida y proporcionan rutas lógicas de decisión (reiniciar con nuevos parámetros o continuar jugando en modo libre).
- **dialogo-opciones / ventana-inicio:** Marcos de diálogo flotantes que agrupan componentes de entrada de texto (text-field%) y botones de confirmación, encargados de la parametrización dinámica del tamaño de la cuadrícula al iniciar la ejecución del código y durante un reinicio forzado.

1.2. Ejemplificación de las estructuras de datos desarrolladas

Para el desarrollo de la lógica, el tablero se abstrae como una lista de listas puramente funcional. Cada sublista interna representa una fila completa del tablero, y los elementos dentro de ellas representan el valor de las casillas en esa fila y una colunma. Un espacio vacío se representa con un 0.

Por ejemplo, un tablero de 2x2 que contiene una ficha con el número "2" en la esquina superior izquierda y otra ficha "2" en la esquina inferior izquierda se representa internamente mediante la siguiente estructura:

```
' ((2 0)
 (0 2) )
```

Esta estructura representa el tablero, donde cada sublista es una casilla del tablero. La primera posición con un 2 indica que la casilla tiene un valor de 2.

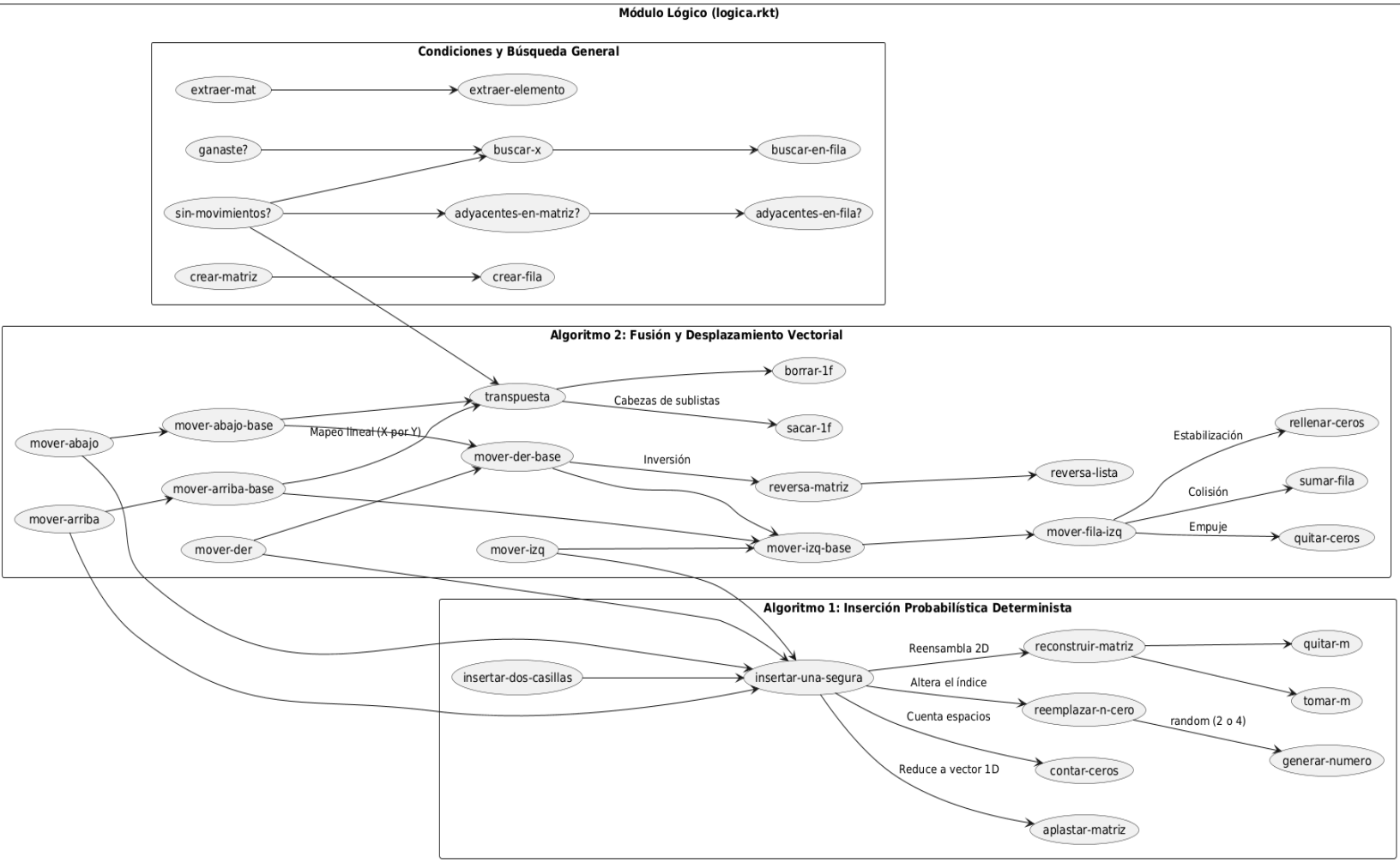
1.3. Descripción detallada de los algoritmos desarrollados

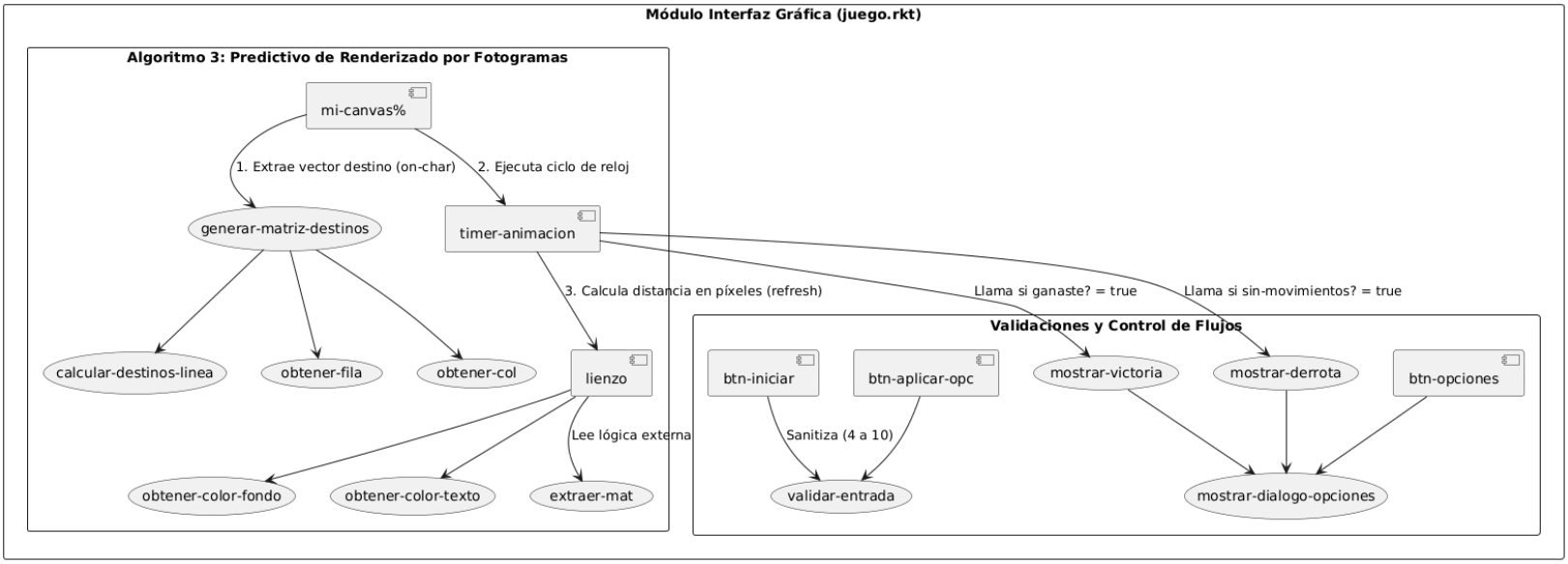
El proyecto demandó el diseño de una arquitectura de software dividida en tres algoritmos robustos principales que interactúan entre sí:

1. **Algoritmo de Inserción Probabilística Determinista:** Inicialmente, el enfoque consistía en seleccionar aleatoriamente coordenadas (X, Y) hasta encontrar un campo vacío. Se desarrolló un algoritmo superior que aúna el aplanamiento y conteo transversal. La estructura n x m se reduce a un vector unidimensional mediante recorridos recursivos (append). Se cuenta el número exacto de ceros disponibles. Un generador estocástico (random) escoge un índice con precisión absoluta, y una función de reemplazo secuencial altera dicho cero por un número ponderado (2 o 4). Finalmente, se reensambla la geometría matricial original. Esto elimina la complejidad de tiempo indefinido, previniendo bucles infinitos en tableros saturados.
2. **Algoritmo de Fusión y Desplazamiento Vectorial:** En lugar de crear cuatro lógicas independientes, el algoritmo condensa la física en un único eje de movimiento (hacia la

izquierda). El proceso despoja a la lista de ceros ("empuje"), compara vecinos mediante composición matemática para fusionar ("colisión") y apila ceros compensatorios ("estabilización estructural"). Para los movimientos verticales, el algoritmo aplica un mapeo lineal que transpone el eje X por el eje Y iterando a través de las cabezas de las sublistas, ejecuta el deslizamiento horizontal normal, y vuelve a transponer la matriz, garantizando integridad funcional y economía de código sin el uso de variables mutables.

3. **Algoritmo Predictivo de Renderizado por Fotogramas:** Para evitar el "efecto fantasma" donde casillas estáticas visualmente atraviesan los límites durante las animaciones, se diseñó un motor de telemetría. Al capturar la pulsación del teclado, el algoritmo extrae un vector de destino calculando prematuramente el índice matemático de reposo de cada ficha basándose en sus sumas proyectadas. Durante la ejecución del ciclo de reloj (timer%), la distancia en píxeles es calculada individualmente para cada bloque mediante el teorema de desplazamiento, logrando que bloques bloqueados tengan velocidad 0, mientras otros colisionan detrás de ellos orgánicamente.





1.4. Problemas sin solución

No se presentaron problemas sin solución. Se logró mitigar todas las limitaciones técnicas encontradas a lo largo de las pruebas. Se implementó exitosamente una arquitectura híbrida que respeta el motor funcional subyacente al tiempo que ofrece una experiencia gráfica de alto nivel, cumpliendo con todos los requerimientos estipulados y logrando todos los objetivos específicos con éxito.

1.5. Plan de Actividades realizadas por estudiante

El desarrollo de este proyecto fue realizado de forma individual por Jestim. El planeamiento original para completar la tarea se dividió en las siguientes actividades principales:

- Crear una función que genere una matriz de n*m, recibe dos parámetros, la n y m, y crearla con 0 en todas las casillas.
- Crear una función para acceder a un index de la matriz.
- Crear una función que reciba una matriz como parámetro y retorne la matriz modificada, de manera aleatoria agarre dos casillas que tengan el valor 0 y le ponga con un 90% de probabilidad un 2 o con un 10% un 4.
- Crear una función que reciba una matriz como parámetro y retorne la matriz modificada, de manera aleatoria agarre una casilla con valor 0 y le ponga con un 90% de probabilidad un 2 o con un 10% un 4.
- Crear una función que recorra la matriz para encontrar un valor “x” y si encuentra ese valor retornar true.
- Funciones para mover elementos de la matriz:
  1. Crear una función que reciba una matriz y la palabra “Arriba”, esto significa que moverá todos los elementos hacia arriba de la siguiente manera; los elementos que

puedan moverse a la primera fila lo harán, los otros elementos dependerá de cual es el valor de ellos, si una casilla colisiona con otra y son el mismo valor se suman, si no solo queda debajo de la casilla, entonces antes de mover una casilla a otra posición se verifica si la colisión se da con una casilla del mismo valor entonces se genera una casilla nueva con el valor de suma de ambas en la posición donde estaba la primera casilla, por ejemplo en 4x4, si una casilla de valor 2 esta en la fila 0, y otra casilla de valor 2 en la fila 1 colisiona con la casilla de valor 2 en la fila 0, se hace una casilla nueva con el valor 4 en la fila 0 en la misma posición que la casilla con valor 2 en la fila 0. Si no se da una colisión de suma, por ejemplo en 4x4, si una casilla de valor 2 esta en la fila 0, y otra casilla de valor 4 en la fila 2 y están en la misma columna la casilla de valor 4 solo quedara abajo de la casilla de valor 2.
2. 3 funciones más harán lo mismo pero hacia “Abajo”, “Izquierda”, “Derecha” y el límite para que las casillas no se salgan de la matriz, será la primera fila para “Arriba”, será la ultima fila para “Abajo”, la ultima columna para “Derecha”, la primera columna para “Izquierda”.
3. Cada vez que se muevan los elementos con esas funciones se utiliza la función para generar una casilla con un nuevo valor.

1.6. Problemas encontrados

Durante la ejecución del plan de actividades principal, se detectaron diversas limitaciones que forzaron alteraciones arquitectónicas importantes:

- **Lógica de Inserción:** El plan original dictaba "agarrar una casilla con valor 0 de manera aleatoria". Se encontró el problema de que en tableros avanzados, el intento de buscar casillas vacías al azar causaba sobrecarga de procesamiento o bucles infinitos. El plan se modificó hacia un algoritmo riguroso de extracción aplanada, el cual cuenta con precisión absoluta los espacios vacíos y garantiza la inserción en O(N) operaciones.
- **Arquitectura de Movimiento:** Originalmente se había planificado codificar tres funciones imperativas adicionales que hicieran "lo mismo pero hacia Abajo, Izquierda y Derecha" gestionando los límites manualmente. Para mantener la estricta adherencia a la programación funcional orientada a listas, la solución encontrada fue introducir operaciones de álgebra lineal matricial (transposición e inversión en espejo) para reutilizar un único motor de movimiento base hacia la izquierda.

1.7. Conclusiones y Recomendaciones del proyecto

Conclusiones:

- Se lograron cumplir exitosamente los objetivos específicos del proyecto desarrollando una réplica funcional del juego 2048 utilizando Racket, integrando una arquitectura modular.

- La aplicación práctica de los conceptos de programación funcional demostró ser altamente eficiente para la manipulación topológica de estructuras de datos bidimensionales (listas de listas).
- Se cumplieron con todos los requerimientos de restricción de operadores, demostrando que la recursividad de cola y las evaluaciones algorítmicas con cond son suficientes para resolver problemas de lógica de juegos sin requerir mutación imperativa en el núcleo del motor.

Recomendaciones:

- Se recomienda, para futuras iteraciones del proyecto, implementar un sistema de Puntuación (Score) que incremente una variable global conforme se fusionan las casillas numéricas.
- Integrar un sistema de lectura y escritura de archivos planos para permitir el almacenamiento de la puntuación máxima histórica (High Score) entre distintas sesiones de juego.
- Diseñar un mecanismo de memoria transaccional tipo pila (Stack) que guarde la matriz del turno anterior para implementar un botón de Deshacer (Undo), proporcionando mayor margen de recuperación al jugador.

1.8. Bibliografía consultada en todo el proyecto

- Guzmán, J. E. (2006). Introducción a la programación con Scheme. Cartago: Editorial Tecnológica de Costa Rica. (Referenciado para estructuras de control recursivas, manejo de listas simples, abstracciones de conjuntos y matrices).
- Wikipedia. (2026, March 18). 2048 (videojuego). Retrieved from Wikipedia: https://es.wikipedia.org/wiki/2048_(videojuego)
- Racket. (2017, 08 15). The Racket Graphical Interface Toolkit. Retrieved from Racket: http://download.racket-lang.org/releases/6.9/doc/gui/

Enlance a github:
https://github.com/JestimEM/TareaRacket.git